

Kuik — Plan de remediación

13 puntos con pasos concretos: 12 hallazgos de código y base de datos, más la capa de infraestructura. Ordenados por urgencia real: los dos primeros son explotables hoy por cualquier persona con un correo.

Alcance: routing multi-tenant, endpoints públicos, server actions, políticas RLS (39 migraciones), storage e infraestructura de despliegue.

Rama: pos-kds · **Stack:** Next.js 16 · Supabase · Render · MercadoPago

Fecha: 24 de agosto de 2026

Orden de ataque

Haz los puntos 1 y 2 hoy. Son una sola migración SQL de ~30 líneas y ahorran la posibilidad de que alguien te tome la plataforma completa. El 3 y el 4 son los que evitan que un script simple tire el servicio.

#	PROBLEMA	SEVERIDAD	DÓNDE SE ARREGLA	TIEMPO
1	Cualquier usuario puede volverse <code>super_admin</code>	CRÍTICO	Migración SQL	5 min
2	Cualquier usuario borra/reemplaza archivos de todos	CRÍTICO	Migración SQL	15 min
3	Inyección de filtros en la consulta de tenant	ALTO	<code>lib/tenant.ts</code>	20 min
4	Endpoints públicos sin límite (y fuga de datos de clientes)	ALTO	2 archivos nuevos + 4 rutas	1–2 h
5	SSRF al importar imágenes por URL	MEDIO	<code>full-import-actions.ts</code>	30 min
6	Secretos que fallan abiertos si falta la variable	MEDIO	2 rutas	10 min
7	Sin cabeceras de seguridad	MEDIO	<code>next.config.ts</code>	15 min
8	Dominios ajenos se pueden registrar en tu servicio	MEDIO	Migración + <code>domain/actions.ts</code>	1 h
9	Registro abierto sin captcha ni verificación	MEDIO	Panel de Supabase	30 min
10	Amplificación de caché con URLs basura	BAJO	Se resuelve con el #3	—
11	Inyección de CSS por nombre de fuente	BAJO	<code>s/[tenant]/layout.tsx</code>	15 min
12	<code>product_views</code> crece sin límite	BAJO	Migración SQL	15 min
13	Sin WAF ni CDN delante del servidor	INFRA	Cloudflare	1–2 h

ANTES DE EMPEZAR: CÓMO APLICAR UNA MIGRACIÓN

Los puntos 1, 2, 8 y 12 son SQL. Puedes juntarlos todos en un solo archivo. El proceso es siempre el mismo:

- 1 Crea el archivo `supabase/migrations/0040_security_hardening.sql` y pega dentro el SQL de cada punto.
- 2 Aplícalo con `npx supabase db push`.
Si no tienes el proyecto enlazado con la CLI: abre el panel de Supabase → **SQL Editor** → **New query** → pega el SQL → **Run**. Guarda igual el archivo en el repo para que quede versionado.
- 3 Ejecuta la consulta de verificación que viene al final de cada punto.

1 **CRÍTICO**

Cualquier usuario puede volverse `super_admin`

QUÉ PASA

La política que deja a cada quien editar su propio perfil no limita *qué columnas* puede tocar. Entonces cualquiera que se registre puede cambiarse la columna `role` a `super_admin` desde la consola del navegador, usando la anon key que es pública. Como `is_super_admin()` aparece en casi todas las políticas RLS del proyecto, eso da acceso total: leer y escribir los datos de todos los restaurantes, entrar a `/admin`, cambiar los precios de la plataforma y regalarse meses gratis.

Origen: `supabase/migrations/0001_init.sql`, líneas 254-257.

PASOS

- 1 Abre `supabase/migrations/0040_security_hardening.sql` y pega esto:

```
-- 1. Nadie puede modificar su propio rol.  
-- No hace falta tocar la política: quitamos el permiso a nivel columna,  
-- que es más fuerte porque aplica pase lo que pase con las policies.  
revoke update (role) on public.profiles from anon, authenticated;
```

- 2 Aplica la migración (`npx supabase db push`).
- 3 Revisa quién es `super_admin` hoy. Si aparece alguien que no reconoces, ya te lo explotaron: bájalo y revisa `audit_log`.

```
select p.id, u.email, p.role  
from public.profiles p join auth.users u on u.id = p.id  
where p.role = 'super_admin';
```

CÓMO VERIFICAR

Entra a la app con una cuenta normal, abre la consola del navegador y ejecuta un `update` sobre tu propio perfil poniendo `role: 'super_admin'`. Debe responder error 42501 – permission denied for column role. Si responde éxito, la migración no se aplicó.

No rompe nada: revisé todo el código y **ninguna parte de la app escribe en `profiles`**. Para promover a alguien a `super_admin` sigues usando `supabase/set_super_admin.sql` desde el SQL Editor, que corre como `postgres` y no se ve afectado.

2 **CRÍTICO**

Cualquier usuario puede borrar o reemplazar los archivos de todos los restaurantes

QUÉ PASA

Las políticas del bucket `media` sólo comprueban `bucket_id = 'media'`, sin mirar la carpeta. Cualquier cuenta autenticada puede escribir, sobrescribir y

BORRAR

cualquier archivo del bucket: logos, fotos de productos, PDFs de menú y las landings personalizadas de todos tus clientes. También puede subir archivos ilimitados (tu factura de Supabase) y hospedar contenido arbitrario en tu dominio de storage.

Origen: `supabase/migrations/0001_init.sql`, líneas 296-303.

PASOS

- 1 Agrega esto a la migración `0040`:

```
-- 2. El primer segmento de la ruta es el tenant_id: "<tenant>/logos/x.jpg".
-- Sólo miembros de ese tenant (o super_admin) pueden escribir ahí.
create or replace function public.media_path_allowed(objname text)
returns boolean language plpgsql stable set search_path = public as $$
declare seg text;
begin
  seg := (storage.foldername(objname))[1];
  -- Si el primer segmento no es un UUID, se rechaza (no se intenta castear).
  if seg is null or seg !~ '^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$'
  then return false; end if;
  return public.is_member(seg::uuid) or public.is_super_admin();
end; $$;

drop policy if exists "media authed write" on storage.objects;
drop policy if exists "media authed update" on storage.objects;
drop policy if exists "media authed delete" on storage.objects;

create policy "media tenant write" on storage.objects for insert to authenticated
with check (bucket_id = 'media' and public.media_path_allowed(name));

create policy "media tenant update" on storage.objects for update to authenticated
using      (bucket_id = 'media' and public.media_path_allowed(name))
with check (bucket_id = 'media' and public.media_path_allowed(name));

create policy "media tenant delete" on storage.objects for delete to authenticated
using (bucket_id = 'media' and public.media_path_allowed(name));
```

- 2 Aplica la migración. **No toques la política de lectura** (`media public read`): los menús son públicos y deben seguir siéndolo.
- 3 Busca archivos que no pertenezcan a ningún tenant (posible basura ya subida):

```
select name, created_at from storage.objects
where bucket_id = 'media' and not public.media_path_allowed(name)
order by created_at desc;
```

CÓMO VERIFICAR

Entra con una cuenta de un restaurante A y sube una imagen desde el dashboard: debe funcionar igual que antes. Luego, desde la consola, intenta subir a la carpeta de un restaurante B (`storage.from('media').upload('<id-de-`

```
B>/logos/x.png', file) ): debe fallar con new row violates row-level security policy .
```

OJO

Las landings personalizadas las sube el super-admin con el cliente service-role, que ignora RLS. Esa función sigue funcionando sin cambios.

3 ALTO

Inyección de filtros en la consulta que resuelve el restaurante

QUÉ PASA

La búsqueda del tenant arma el filtro pegando texto:

`.or('subdomain.eq.${hostKey}, custom_domain.eq.${hostKey}')`. Ese `hostKey` sale del header `Host` y también, directamente, de la URL: `/s/lo-que-sea` es una ruta real y alcanzable. Como no se valida, se pueden meter operadores dentro del filtro (`/s/x, subdomain.like.*`) para servir restaurantes arbitrarios y, sobre todo, forzar escaneos completos de tabla sin autenticación — munición barata para tumbar la base de datos.

Origen: `lib/tenant.ts` línea 34 · `proxy.ts` línea 33 · mismo patrón en `app/(dashboard)/loyalty/actions.ts` línea 24.

PASOS

- 1 En `lib/tenant.ts`, arriba del archivo, agrega la validación:

```
// Un host válido: letras, números, guiones y puntos. Nada más.
const HOST_KEY_RE = /^[a-z0-9]([a-z0-9-]{0,61}[a-z0-9])?(\.[a-z0-9]([a-z0-9-]{0,61}[a-z0-9])?)*$/;
```

- 2 Dentro de `getTenantByHostKey`, reemplaza el bloque del `.or(...)` por esto:

```
const key = hostKey.toLowerCase();
if (key.length > 253 || !HOST_KEY_RE.test(key)) return null;

const supabase = createAdminClient();

// Dos consultas con .eq() en vez de un .or() interpolado.
let { data: tenant } = await supabase
  .from('tenants').select('*').eq('subdomain', key).maybeSingle<Tenant>();

if (!tenant) {
  ({ data: tenant } = await supabase
    .from('tenants').select('*').eq('custom_domain', key).maybeSingle<Tenant>());
}

if (!tenant) return null;
```

- 3 En `app/(dashboard)/loyalty/actions.ts`, función `findCustomer`, aplica lo mismo:

```
const code = q.toUpperCase();
const phone = q.replace(/\D/g, '');

// Busca por código si tiene forma de código; si no, por teléfono.
let data = null;
if (/^[A-Z0-9]{1,12}$/.test(code)) {
  ({ data } = await supabase.from('loyalty_customers').select('*')
    .eq('tenant_id', tenantId).eq('code', code).maybeSingle<LoyaltyCustomer>());
}
if (!data && phone.length >= 8) {
  ({ data } = await supabase.from('loyalty_customers').select('*')
    .eq('tenant_id', tenantId).eq('phone', phone).maybeSingle<LoyaltyCustomer>());
}
return data ?? null;
```

- 4 Busca en el resto del código si queda algún otro `.or(` con texto interpolado:

```
grep -rn "\.or(\" app lib components
```

CÓMO VERIFICAR

Abre `https://app.kuik.mx/s/x,subdomain.like.*` — debe devolver 404, no un menú. Y confirma que un subdominio normal (`tacos.kuik.mx`) y un dominio propio siguen cargando bien.

4 ALTO

Endpoints públicos sin ningún límite (y fuga de datos de clientes)

QUÉ PASA

Los cuatro endpoints anónimos escriben directo con el service-role (saltándose RLS), sin límite de peticiones, sin tope de tamaño del cuerpo y sin comprobar siquiera que el restaurante exista.

- `/api/track` — inserciones ilimitadas en `product_views` : inflar la base hasta reventar el plan.
- `/api/order` — el campo `items` no tiene tope: un POST de 50 MB por petición.
- `/api/reservation` — spam ilimitado de reservas al dashboard del restaurante.
- `/api/loyalty` —

FUGA DE DATOS:

mandas un teléfono y devuelve nombre, sellos y puntos de ese cliente. Cualquiera puede recorrer rangos de teléfonos y extraer la base de clientes completa de un restaurante.

Origen: `app/api/{track,order,reservation,loyalty}/{tenantId}/route.ts`.

PASOS

- 1 Crea `lib/rate-limit.ts` (limitador en memoria; sin infraestructura nueva):

```
const hits = new Map<string, { count: number; reset: number }>();

/** true = permitido, false = pasó del límite. */
export function rateLimit(key: string, limit: number, windowMs: number): boolean {
  const now = Date.now();

  // Limpieza perezosa para que el Map no crezca sin control.
  if (hits.size > 10_000) {
    for (const [k, v] of hits) if (now > v.reset) hits.delete(k);
  }

  const e = hits.get(key);
  if (!e || now > e.reset) { hits.set(key, { count: 1, reset: now + windowMs }); return true; }
  if (e.count >= limit) return false;
  e.count++;
  return true;
}
```

- 2 Crea `lib/api-guard.ts` con las tres comprobaciones que faltan:

```
import { NextResponse, type NextRequest } from 'next/server';
import { rateLimit } from './rate-limit';

const UUID_RE = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;

export function clientIp(req: NextRequest): string {
  return req.headers.get('x-forwarded-for')?.split(',')[0].trim() ?? 'unknown';
}

/** Valida tenant, límite por IP y tamaño del cuerpo. Devuelve el JSON o un error. */
export async function guard(
  req: NextRequest,
  tenantId: string,
  o: { name: string; limit: number; windowMs: number; maxBytes: number },
): Promise<{ error: NextResponse } | { body: unknown }> {
```



```
const deny = (s: number) => ({ error: NextResponse.json({ ok: false }, { status: s }) });

if (!UUID_RE.test(tenantId)) return deny(400);
if (!rateLimit(`${o.name}:${clientIp(req)}`, o.limit, o.windowMs)) return deny(429);
if (Number(req.headers.get('content-length')) ?? 0) > o.maxBytes) return deny(413);

const text = await req.text();
if (text.length > o.maxBytes) return deny(413);

try { return { body: JSON.parse(text) }; } catch { return deny(400); }
}
```

- 3 En cada ruta, sustituye el `try { body = await req.json() }` por el guard. Ejemplo con `/api/track`:

```
const { tenantId } = await params;

const g = await guard(req, tenantId, {
  name: 'track', limit: 60, windowMs: 60_000, maxBytes: 2_000,
});
if ('error' in g) return g.error;

const { productId } = g.body as { productId?: string };
if (!productId) return NextResponse.json({ ok: false }, { status: 400 });
```

Usa estos números en cada una:

RUTA	LÍMITE POR IP	TAMAÑO MÁX.
<code>/api/track</code>	60 / minuto	2 KB
<code>/api/order</code>	10 / minuto	64 KB
<code>/api/reservation</code>	5 / minuto	4 KB
<code>/api/loyalty</code>	5 / minuto	1 KB

- 4 Añade un tope duro a los pedidos, en `/api/order`, antes de insertar:

```
if (!Array.isArray(body.items) || body.items.length === 0 || body.items.length > 200) {
  return NextResponse.json({ ok: false }, { status: 400 });
}
```

- 5 **Cierra la fuga de `/api/loyalty`**. Hoy devuelve los datos de un cliente existente sin ninguna prueba de que el teléfono es tuyo. Elige una:
- **Rápido (hoy):** si el cliente *ya existe*, no devuelvas `name` ni el historial completo — devuelve sólo el `code` y los sellos. Reduce el valor de raspar la base, aunque no lo elimina.
 - **Correcto (esta semana):** exige un código de 6 dígitos enviado por WhatsApp antes de devolver la tarjeta. Es el único arreglo real: el teléfono es la identidad, así que hay que probar que lo posees.

CÓMO VERIFICAR

```
for i in $(seq 1 20); do curl -s -o /dev/null -w "%{http_code} " -X POST
https://app.kuik.mx/api/reservation/<tenant-id> -d '{}'; done — a partir de la sexta petición debe
devolver 429 .
```

LÍMITE DE ESTE ENFOQUE

El limitador vive en la memoria del proceso: si algún día escalas a varias instancias en Render, cada una tendrá su propio contador. Funciona bien con una instancia y es el 90% del beneficio por el 10% del trabajo. Cuando escales, mueve el

contador a Upstash Redis o apóyate en el rate limiting de Cloudflare (punto 13).

SSRF al importar imágenes por URL

QUÉ PASA

Al importar un menú, `resolveImage()` hace `fetch()` de cualquier URL que le pases y sube la respuesta al storage público, devolviéndote el enlace. Eso permite que el servidor consulte por ti direcciones internas de la red de Render o servicios sin autenticación, y que el contenido te llegue de vuelta. Requiere ser manager de algún restaurante — es decir, cualquiera que se registre.

Origen: `app/(dashboard)/menu/full-import-actions.ts`, líneas 52–64.

PASOS

- 1 Al inicio de `full-import-actions.ts`, agrega el validador:

```
import { lookup } from 'node:dns/promises';

// Rangos privados, loopback y link-local (incluye 169.254.169.254, metadata).
const PRIVATE_IP =
  /^(0\.|10\.|127\.|169\.254\.|192\.168\.|172\.(1[6-9]|2\d|3[01])\.\.::1$|fc|fd|fe80)/i;

const MAX_IMAGE_BYTES = 5 * 1024 * 1024;

async function isSafeUrl(raw: string): Promise<boolean> {
  let u: URL;
  try { u = new URL(raw); } catch { return false; }
  if (u.protocol !== 'https:' && u.protocol !== 'http:') return false;
  try {
    const { address } = await lookup(u.hostname);
    return !PRIVATE_IP.test(address);
  } catch { return false; }
}
```

- 2 Dentro de `resolveImage()`, justo después del `if (supaUrl && ref.startsWith(supaUrl)) return ref;`, añade la comprobación y los topes:

```
if (!(await isSafeUrl(ref))) return null;

const res = await fetch(ref, { redirect: 'error' }); // sin redirecciones
if (!res.ok) return null;

const ct = res.headers.get('content-type') ?? '';
if (!ct.startsWith('image/')) return null; // sólo imágenes

const buf = await res.arrayBuffer();
if (buf.byteLength > MAX_IMAGE_BYTES) return null; // tope de 5 MB
const bytes = new Uint8Array(buf);
```

- 3 El `redirect: 'error'` es importante: sin eso, una URL pública puede redirigir a una dirección interna y saltarse la validación.

CÓMO VERIFICAR

Importa un menú con una imagen apuntando a `http://169.254.169.254/latest/meta-data/` y a `http://127.0.0.1:3000/`. En ambos casos el producto debe quedar sin imagen, no con un archivo subido.

6 MEDIO

Secretos que fallan abiertos si falta la variable de entorno

QUÉ PASA

El webhook de MercadoPago y el cron comprueban `if (secret && ...)`. Si la variable no está configurada en el despliegue, la condición se salta entera y el endpoint queda

ABIERTO A INTERNET EN SILENCIO

. Debe ser al revés: sin secreto, no se atiende a nadie.

Origen: `app/api/webhooks/mercadopago/route.ts` línea 28 · `app/api/cron/expire-trials/route.ts` línea 14.

PASOS

- 1 En el webhook de MercadoPago, reemplaza el bloque del secreto:

```
const secret = process.env.MERCADOPAGO_WEBHOOK_SECRET;
if (!secret) {
  console.error('[mercadopago] falta MERCADOPAGO_WEBHOOK_SECRET');
  return NextResponse.json({ ok: false }, { status: 500 });
}
if (req.nextUrl.searchParams.get('secret') !== secret) {
  return NextResponse.json({ ok: false }, { status: 401 });
}
```

- 2 Lo mismo en el cron:

```
const secret = process.env.CRON_SECRET;
if (!secret) return NextResponse.json({ ok: false }, { status: 500 });
if (req.headers.get('authorization') !== `Bearer ${secret}`) {
  return NextResponse.json({ ok: false }, { status: 401 });
}
```

- 3 En el panel de Render, confirma que `MERCADOPAGO_WEBHOOK_SECRET` y `CRON_SECRET` están puestos **en los dos servicios** (el web y el cron job).

CÓMO VERIFICAR

`curl -i https://app.kuik.mx/api/cron/expire-trials` sin cabecera debe dar `401`. Y revisa al día siguiente que el cron de Render sigue corriendo en verde.

7 MEDIO

Faltan las cabeceras de seguridad

QUÉ PASA

`next.config.ts` no define ninguna cabecera. No hay protección contra clickjacking (alguien puede meter tu dashboard en un iframe invisible y hacer que tus usuarios hagan clic donde no quieren), ni HSTS, ni control de referer.

Origen: `next.config.ts`.

PASOS

- 1 Dentro de `nextConfig`, junto a `images`, agrega:

```
async headers() {
  return [
    {
      // Sólo el dashboard: prohibido embeberlo en cualquier iframe.
      source: '/*:path*',
      has: [{ type: 'host', value: 'app.kuik.mx' }],
      headers: [
        { key: 'X-Frame-Options', value: 'DENY' },
        { key: 'Content-Security-Policy', value: "frame-ancestors 'none'" },
      ],
    },
    {
      // Todo el resto del sitio, menús de clientes incluidos.
      source: '/*:path*',
      headers: [
        { key: 'Strict-Transport-Security', value: 'max-age=63072000; includeSubDomains' },
        { key: 'X-Content-Type-Options', value: 'nosniff' },
        { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
        { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=(), payment=()' },
      ],
    },
  ];
}
```

- 2 Nota por qué la separación: los menús de tus clientes **sí** deben poder embeberse (un restaurante puede querer meter su menú en su propia web). El dashboard, no.
- 3 Una CSP completa (que controle scripts y estilos) es un proyecto aparte, porque Next inyecta scripts y estilos en línea. Cuando lo hagas, empieza con `Content-Security-Policy-Report-Only` durante una semana y revisa qué se rompería antes de activarla de verdad.

CÓMO VERIFICAR

`curl -sI https://app.kuik.mx/login | grep -i "x-frame\|strict-transport\|referrer"` debe listar las tres. Y confirma que la landing personalizada de un cliente (que va en un iframe) sigue viéndose bien.

8 MEDIO

Se pueden registrar dominios ajenos en tu servicio de Render

QUÉ PASA

`connectDomain()` registra en tu servicio de Render cualquier dominio que le manden, sin comprobar que quien lo pide realmente lo controla. Alguien puede ocupar dominios ajenos y quemar la cuota de dominios de tu servicio.

Origen: `app/(dashboard)/domain/actions.ts`, línea 29.

PASOS

- 1 Añade a la migración `0040` un token por restaurante:

```
alter table tenants add column if not exists domain_verify_token text
not null default encode(gen_random_bytes(16), 'hex');
```

- 2 En `domain/actions.ts`, antes de llamar a `addRenderDomain`, comprueba el registro TXT:

```
import { resolveTxt } from 'node:dns/promises';

try {
  const records = await resolveTxt(`_kuik.${domain}`);
  const ok = records.flat().includes(`kuik-verify=${tenant.domain_verify_token}`);
  if (!ok) return { error: 'notVerified' };
} catch {
  return { error: 'notVerified' };
}
```

- 3 En la pantalla de dominio (`components/dashboard/DomainManager.tsx`), muestra al usuario el registro que tiene que crear antes de darle al botón:

```
Tipo: TXT
Nombre: _kuik
Valor: kuik-verify=<domain_verify_token>
```

- 4 Agrega el mensaje `notVerified` a `messages/es.json` y `messages/en.json`.

CÓMO VERIFICAR

Intenta conectar un dominio que no controles (por ejemplo `google.com`): debe rechazarlo. Luego haz el flujo completo con un dominio real tuyo y confirma que, tras crear el TXT, sí conecta.

9

MEDIO

Registro abierto sin captcha ni verificación de correo

QUÉ PASA

Cualquier bot puede crear cuentas en masa. Cada cuenta creada es una llave de escritura al storage (ver punto 2) y crea filas de restaurante, subdominio y suscripción. Esto se arregla casi todo desde el panel de Supabase, sin tocar código.

PASOS

- 1 Panel de Supabase → **Authentication** → **Sign In / Providers** → Email → activa **Confirm email**. Con esto, una cuenta sin correo verificado no obtiene sesión y no puede crear nada.
- 2 Crea un sitio en **Cloudflare Turnstile** (es gratis) y copia la site key y la secret key.
- 3 Panel de Supabase → **Authentication** → **Attack Protection** → activa **Enable Captcha protection**, elige Turnstile y pega la secret key.
- 4 En la misma sección, baja los límites de **Rate Limits** para signup y para envío de correos (los valores por defecto son generosos).
- 5 En el formulario de registro (`app/(auth)/signup/page.tsx`), añade el widget de Turnstile y pasa el token a la acción; en `app/(auth)/actions.ts` :

```
const { data, error } = await supabase.auth.signUp({
  email,
  password,
  options: {
    data: { full_name: fullName },
    captchaToken: String(formData.get('captchaToken') ?? ''),
  },
});
```

- 6 Añade `NEXT_PUBLIC_TURNSTILE_SITE_KEY` a `.env.local`, a `.env.example` y a las variables del servicio en Render.

CÓMO VERIFICAR

Regístrate con un correo nuevo: debes recibir el correo de confirmación y no poder entrar hasta hacer clic. Y si envías el formulario sin resolver el captcha, Supabase debe rechazarlo con `captcha protection: request disallowed`.

10

BAJO

Amplificación de caché con URLs basura

QUÉ PASA

Cada `/s/<clave-inventada>` genera una entrada de caché nueva y una consulta a la base. Sin validación, es munición gratis para saturar memoria y base de datos.

PASOS

- 1 **Ya queda resuelto con el punto 3:** al validar el formato del host antes de consultar, las claves basura se rechazan sin tocar la base de datos.
- 2 Complétalo con la caché de Cloudflare del punto 13, para que los menús válidos ni siquiera lleguen al servidor.

11

BAJO

Inyección de CSS por el nombre de la fuente

QUÉ PASA

`font_family` y `custom_font_url` se pegan tal cual dentro de una etiqueta de estilos. React escapa los signos `<`, así que

NO HAY XSS

, pero un restaurante sí puede inyectar CSS arbitrario en su propia página. Es de riesgo bajo, pero se arregla en 15 minutos.

Origen: `app/s/[tenant]/layout.tsx`, línea 170.

PASOS

- 1 En `layout.tsx`, valida antes de usar el valor:

```
import { MENU_FONTS, CUSTOM_FONT } from '@lib/config';

const ALLOWED = new Set<string>([...MENU_FONTS, CUSTOM_FONT]);

function fontCss(value: string): string {
  return ALLOWED.has(value) ? `${value}` : "'Inter'";
}
```

- 2 Y valida la URL de la fuente subida antes de meterla en el `@font-face`:

```
const supaUrl = process.env.NEXT_PUBLIC_SUPABASE_URL ?? '';
const fontUrl =
  theme.custom_font_url?.startsWith(supaUrl) ? theme.custom_font_url : null;
```

Después usa `fontUrl` en lugar de `theme.custom_font_url`.

CÓMO VERIFICAR

Los menús con fuentes normales y con fuente subida siguen viéndose igual. Pon a mano un `font_family` inválido en la base y confirma que cae a Inter en vez de aplicar CSS raro.

12

BAJO

product_views **crece sin límite**

QUÉ PASA

La tabla de analítica no se purga nunca. Con el tiempo (y sobre todo si alguien abusa de `/api/track`) domina el tamaño de la base y te saca del plan.

PASOS

- 1 Agrega a la migración `0040` la purga automática:

```
create extension if not exists pg_cron;

create or replace function public.purge_product_views()
returns void language sql security definer set search_path = public as $$
    delete from product_views where viewed_at < now() - interval '90 days';
$$;

select cron.schedule(
    'purge-product-views',
    '30 5 * * *',                -- 05:30 UTC, diario
    $$select public.purge_product_views()$$
);
```

- 2 Si prefieres no usar `pg_cron`, mete la misma sentencia `delete` dentro de la ruta `/api/cron/expire-trials`, que ya corre a diario.
- 3 Limpia lo acumulado hoy (una sola vez):

```
delete from product_views where viewed_at < now() - interval '90 days';
```

Sin WAF ni CDN delante del servidor

QUÉ PASA

Nada de lo anterior te protege de un ataque por volumen. Hoy estás en `Render starter` : una sola instancia, con el tráfico llegando directo. Cualquiera con un script de tres líneas y suficiente ancho de banda te tira el servicio. Esta es la capa que de verdad resuelve el DDoS.

PASOS

- 1 Crea una cuenta en Cloudflare y añade `kuik.mx` . Cambia los nameservers del dominio a los que te dé Cloudflare.
- 2 En **DNS**, pon en **modo proxy** (la nube naranja) los registros de `kuik.mx` , `www` , `app` y el comodín `*` de los subdominios de restaurantes. Con la nube naranja activa, tu IP de origen queda oculta y Cloudflare absorbe los ataques de volumen de forma gratuita.
- 3 **Security** → **Bots** → activa **Bot Fight Mode** (gratis).
- 4 **Security** → **WAF** → **Rate limiting rules**. Crea dos reglas:
 - `/api/*` → máximo 60 peticiones por minuto por IP → bloquear 1 minuto.
 - `/login` y `/signup` → máximo 10 por minuto por IP → challenge.
- 5 **Caching** → **Cache Rules**: cachea en el edge las páginas de menú de los subdominios de restaurante. Ya son ISR de 60 segundos, así que un pico de tráfico legítimo (o un ataque) nunca llega a Supabase.
- 6 En Render, sube el servicio a un plan con autoescalado, o al menos configura varias instancias. Recuerda que si escalas, el limitador del punto 4 deja de ser global — apóyate en el rate limiting de Cloudflare.
- 7 En Supabase, configura alertas de uso de storage, de filas y de ancho de banda, para enterarte del abuso antes que de la factura.

IMPORTANTE CON LOS DOMINIOS PROPIOS DE CLIENTES

Los dominios personalizados de tus restaurantes apuntan directo a Render, así que no pasan por Cloudflare. Si quieres cubrirlos también, tendrás que usar Cloudflare for SaaS (Custom Hostnames) en lugar de registrar cada dominio en Render.

Checklist

Imprime esta página y ve tachando. El orden importa.

HOY — BLOQUEA LA TOMA DE LA PLATAFORMA

- ☐ Revocar `update (role)` en `profiles`
- ☐ Políticas del bucket `media` atadas al tenant
- ☐ Revisar quién es `super_admin` hoy
- ☐ Buscar archivos huérfanos en `media`

ESTA SEMANA — EVITA QUE TE TIREN EL SERVICIO

- ☐ Validar el host en `getTenantByHostKey`
- ☐ Aplicar el guard a las 4 rutas públicas
- ☐ Quitar los `.or()` interpolados
- ☐ Tope de 200 ítems en `/api/order`
- ☐ Crear `lib/rate-limit.ts`
- ☐ Cerrar la fuga de datos de `/api/loyalty`
- ☐ Crear `lib/api-guard.ts`
- ☐ Cloudflare en modo proxy
- ☐ Rate limiting rules en Cloudflare

ESTE MES — CIERRA EL RESTO

- ☐ Arreglar el SSRF de la importación
- ☐ Captcha en el registro
- ☐ Secretos que fallan cerrados
- ☐ Lista blanca de fuentes
- ☐ Cabeceras de seguridad
- ☐ Purga de `product_views`
- ☐ Verificación TXT de dominios
- ☐ Alertas de uso en Supabase
- ☐ Confirmación de correo en Supabase

LO QUE YA ESTÁ BIEN HECHO

Para que quede el balance completo, esto no hace falta tocarlo:

- La service-role key nunca llega al navegador (`lib/supabase/admin.ts` tiene `server-only`).
- RLS está habilitado en todas las tablas, sin excepciones.
- El iframe de las landings personalizadas está aislado **sin** `allow-same-origin`: el JS del cliente corre en un origen opaco y no puede tocar tu sesión ni tus cookies. Es exactamente lo correcto.
- El proxy de landings bloquea el path traversal y manda `X-Content-Type-Options: nosniff`.
- El descomprimido del ZIP filtra `..` y los archivos ocultos.
- El trigger `guard_updated_at` del POS resuelve los conflictos de escritura offline de forma sana.
- La verificación de sesión está bien separada: `getSession()` rápido en el proxy y verificación real de firma con `getClaims()` en cada página.